

Tutorial Sheet: Application Development with Python

Evrad KAMTCHOUM

2024-2025

Module 1: Introduction to Python Programming Language

Objectives:

- Understand syntax, data types, control structures, and functions.
- Write simple programs using built-in functions and modules.

Review Questions:

1. What are the key differences between Python lists and tuples?
2. Explain the concept of mutability in Python.
3. How does Python handle variable scope?

Exercise: Write a program that:

- Takes a list of numbers from the user.
- Filters and prints only the even numbers.
- Uses a function to compute the sum of those numbers.

Module 2: Object-Oriented Programming in Python

Objectives:

- Understand classes, objects, inheritance, encapsulation, and polymorphism.

Review Questions:

1. What is the difference between a class and an object?
2. How does Python support multiple inheritance?
3. What is a static method? Give examples.

Exercise: Create a class `BankAccount` with:

- Attributes: account number, owner name, balance
- Methods: deposit, withdraw, show balance

Then create a subclass `SavingsAccount` that adds interest.

Module 3: Web Application Development with Python (1)

Objectives:

- Understand basic web architecture.
- Set up a simple web app using Flask or Django.
- Learn basic web security principles (HTTPS, headers).

Review Questions:

1. Compare Flask and Django in terms of flexibility and complexity.
2. What are common HTTP methods and their uses?
3. How does CSRF protection work in a web framework?

Exercise: Create a simple Flask application with two routes:

- Home page
- Contact form (GET and POST)

Module 4: Web Development with Python (2)

Objectives:

- Secure coding practices: input validation, sanitization.
- Session management (cookies, tokens, timeouts).

Review Questions:

1. What are the risks of not validating user input?
2. How can XSS and SQL injection be prevented?
3. What is the difference between session and token-based authentication?

Exercise:

- Add form validation to the contact form from Module 3.
- Implement a login form using sessions.

Module 5: Authentication and Authorization in Python Applications

Objectives:

- Implement user authentication securely.
- Use hashing, secure storage, and communication protocols.

Review Questions:

1. What is the difference between authentication and authorization?
2. What are secure ways to store passwords in Python?
3. How does HTTPS secure communication?

Exercise: Add a login/register page to your Flask app using hashed passwords with `werkzeug.security`. Ensure only logged-in users can access a "dashboard" page.

Module 6: Introduction to Application Security Testing

Objectives:

- Debug Python apps and test for vulnerabilities.
- Integrate basic security tools (e.g., Bandit, pytest).

Review Questions:

1. What are the advantages of static vs dynamic analysis tools?
2. How does Bandit help in Python security?
3. What are common test case scenarios for user authentication?

Exercise:

- Install and run Bandit on your project folder.
- Fix any issues it flags.

Module 7: Using Git and GitHub for Python Applications and Projects

Objectives:

- Understand version control with Git.
- Collaborate on projects using GitHub.

Review Questions:

1. What is the purpose of branches in Git?
2. How do you resolve merge conflicts?
3. How does GitHub support collaboration?

Exercise:

- Create a local Git repository for your Flask project.
- Push it to GitHub and write a clear README.md.
- Create a new branch and add a new feature.

Module 8: Introduction to System Programming with Python

Objectives:

- Understand system-level interactions (files, processes, OS).
- Use modules like `os`, `sys`, `subprocess`, and `shutil`.

Review Questions:

1. What are the differences between `os.system()` and `subprocess.run()`?
2. How can you list all files in a directory in Python?
3. What are use cases for Python in system automation?

Exercise: Write a script to:

- List all `.py` files in a directory
- Compress them into a `.zip` file
- Log the operation time and status

Mini Project Prompt

Build a secure mini Flask application for managing a to-do list.

- User registration and login (with hashed passwords)
- CRUD for tasks
- Input validation and session control
- Git version control and README documentation